

torch2rtl: руководство по проекту

От PyTorch-модели к SystemVerilog-описанию

Редактируемый источник: docs/torch2rtl_guide.md

Итоговый файл: docs/torch2rtl_guide.pdf

Подготовлено по текущему состоянию репозитория

17 августа 2026

Содержание

1	Сокращения и обозначения	2
2	Краткое описание	3
3	Задача проекта	3
4	Общая схема преобразования	4
5	Структура каталогов	5
6	Главные модули	5
6.1	Автоматический fusion BatchNorm2d	6
7	Пошаговый разбор демонстрационного примера	9
8	Фиксированная точка	10
9	Создаваемые SystemVerilog-модули	12
10	Установка и запуск	14
11	Проверка результата	15
12	Проведение демонстрации	16
13	Ограничения	16
14	Дальнейшее развитие	18

1 Сокращения и обозначения

Обозначение	Значение
PyTorch	Библиотека Python для описания и обучения нейронных сетей.
torch.fx	Подсистема PyTorch, которая строит граф операций Python-модели.
FX	Часть имени torch.fx; в этом документе используется только в составе имени библиотеки и ее графа.
RTL	Register Transfer Level, уровень описания цифровой схемы через регистры и передачи между ними. В проекте это слово используется для сгенерированных SystemVerilog-файлов, хотя текущая схема в основном комбинационная.
SystemVerilog	Язык описания и проверки цифровых схем.
IR	Intermediate Representation, внутреннее представление модели после разбора графа torch.fx. В коде проекта главный класс называется GraphIR.
EDA	Electronic Design Automation, инструменты для моделирования, анализа и синтеза цифровых схем. В проекте используются Icarus Verilog, Verilator и Yosys, если они установлены.
MAC	Умножение с накоплением: multiply-accumulate. Такая операция лежит в основе слоев Linear и Conv2d.
CLI	Command Line Interface, интерфейс командной строки. В проекте это команда torch2rtl.
JSON	Текстовый формат структурированных данных. В проекте используется для report.json, vectors.json и visualization.json.
HTML	Формат страницы для браузера. В проекте итоговая визуализация записывается в visualization.html.
PDF	Формат переносимого документа. Этот файл собирается из docs/torch2rtl_guide.md.
ПЛИС	Программируемая логическая интегральная схема. Проект пока не выполняет загрузку на плату, но готовит понятные файлы для дальнейших опытов с цифровыми схемами.
Тензор	Массив чисел с формой. Например, вход tiny-conv имеет форму (1, 3, 3).
Логиты	Числа на выходе классификатора до выбора номера класса.
Квантование	Перевод вещественных чисел в целые числа заданной разрядности.
Фиксированная точка	Способ хранить дробные значения в целых числах через общий масштаб. В проекте по умолчанию используются 8 бит и 6 дробных битов.

Обозначение	Значение
Насыщение	Ограничение результата диапазоном выбранной разрядности. Для 8 бит это диапазон от -128 до 127.

2 Краткое описание

torch2rtl берет небольшую модель PyTorch, строит для нее граф `torch.fx`, переводит граф во внутреннее представление, квантует веса и проверочные входы, а затем создает набор SystemVerilog-файлов. Вместе с описанием схемы создаются проверочные данные, отчет, файл для визуализации и проверочный модуль.

Проект не пытается быть промышленным средством автоматического проектирования. Его сильная сторона в другом: вся цепочка короткая, читаемая и пригодная для учебного разбора. Можно увидеть, как слой `torch.nn.Conv2d` или `torch.nn.Linear` становится набором вложенных циклов и сигналов, как фиксированная точка влияет на арифметику, и какие файлы нужны для простой проверки.

Фактическая версия пакета указана в `pyproject.toml` и `torch2rtl/__init__.py` как 0.2.0. Зависимости из `pyproject.toml`: `jinja2`, `numpy`, `torch`; для разработки добавлен `pytest`. Для удобного запуска на обычном ноутбуке `torch` берется из CPU-индекса PyTorch через `tool.uv.sources`, поэтому установка не тянет CUDA-зависимости. Точка входа командной строки объявлена в `pyproject.toml` так:

```
torch2rtl = "torch2rtl.cli:main"
```

3 Задача проекта

Задача проекта: показать воспроизводимый путь от небольшой нейронной сети на PyTorch до SystemVerilog-описания, которое можно открыть, смоделировать и отдать в простой проход Yosys.

Из кода и тестов подтверждены такие поддерживаемые операции:

- `torch.nn.Linear` только для 1-D вектора признаков;
- `torch.nn.ReLU`;
- `torch.nn.Flatten`, только полное преобразование в один вектор;
- `torch.nn.Conv2d` без размерности пакета данных, с обычными группами `groups=1`, нулевым дополнением и `dilation=1`;
- точная пара `torch.nn.Conv2d`->`torch.nn.BatchNorm2d`, которую frontend

автоматически сворачивает в `Conv2d` до `GraphIR`;

- финальный глобальный `argmax(dim=None, keepdim=False)`.

FX-граф должен быть одной последовательной цепочкой с одним входом и одним тензорным выходом. Точные границы подмножества зафиксированы в `docs/v0.2_semantic_correctness.md`.

Ключевое проектное решение: сначала строится маленькое внутреннее представление, а уже потом из него создаются фиксированно-точечная эталонная модель, SystemVerilog, проверочные векторы, отчет и визуализация. Поэтому новые операции нужно добавлять не во все места сразу хаотично, а последовательно: разбор Py-

Torch, внутреннее представление, квантование, эталонный расчет, шаблон SystemVerilog, тесты.

4 Общая схема преобразования

Схема текущей цепочки:

```

Модель PyTorch
-> граф torch.fx
-> fusion пары Conv2d + BatchNorm2d при наличии допустимой пары
-> GraphIR
-> квантованный граф
-> SystemVerilog-файлы
-> input_vectors.txt, expected_classes.txt и expected_logits.txt
-> моделирование через Icarus Verilog или Verilator
-> простой проход Yosys
-> visualization.json и visualization.html

```

Фрагмент из torch2rtl/cli.py показывает, что команда compile делает именно эти первые шаги: собирает конфигурацию фиксированной точки, загружает модель, разбирает ее и вызывает генератор SystemVerilog.

```

def cmd_compile(args: argparse.Namespace) -> int:
    cfg = FixedPointConfig(
        bits=args.bits,
        frac_bits=args.frac_bits,
        acc_bits=args.acc_bits,
    )
    model = load_model_from_file(args.model_path)
    graph = parse_model(model, tuple(args.input_shape))
    report = emit_systemverilog(
        graph=graph,
        cfg=cfg,
        out_dir=args.out,
        vector_count=args.vectors,
        seed=args.seed,
    )

```

В демонстрационном режиме после генерации дополнительно запускаются моделирование, синтез и обновление визуализации. Это видно в torch2rtl/demo.py.

```

build_report = emit_systemverilog(
    graph=graph,
    cfg=cfg,
    out_dir=out_dir,
    vector_count=vector_count,
    seed=vector_seed,
)
update_report_metadata(
    out_dir,
    _demo_report_payload(
        spec=spec,
        out_dir=out_dir,
        cfg=cfg,
        vectors=vector_count,
        seed=vector_seed,
        vector_index=vector_index,
    ),

```

```

)
simulation = run_simulation(out_dir)
synthesis = run_yosys(out_dir)
visualization_path = render_visualization_from_build(
    build_dir=out_dir,
    vector_index=vector_index,
)

```

Почему так устроено: `compile` остается узкой командой, которая только создает артефакты, а `demo` связывает эти артефакты с проверками и отчетом. Это удобно для разработки: можно отдельно проверять генератор, отдельно моделирование и отдельно страницу визуализации.

5 Структура каталогов

Путь	Назначение
README.md	Краткое описание, быстрый запуск, список возможностей и ограничений.
pyproject.toml	Метаданные пакета, зависимости, точка входа <code>torch2rtl</code> , настройки <code>pytest</code> .
torch2rtl/cli.py	Команды <code>compile</code> , <code>verify</code> , <code>synth</code> , <code>visualize</code> , <code>demo</code> .
torch2rtl/demo.py	Описание готовых демонстраций и общий демонстрационный запуск.
torch2rtl/frontend/pytorch_fx.py	Разбор PyTorch-модели через <code>torch.fx</code> и построение GraphIR.
torch2rtl/ir/	Классы внутреннего представления: граф, тензор, операции.
torch2rtl/quant/	Фиксированная точка, квантование и эталонный расчет на Python.
torch2rtl/backend/systemverilog/	Генерация SystemVerilog и шаблоны Jinja2.
torch2rtl/verify/	Проверочные векторы, запуск симуляторов, сравнение статусов.
torch2rtl/synth/	Запуск Yosys и извлечение простых метрик из журнала.
torch2rtl/visualization/	Построение <code>visualization.json</code> и самодостаточной HTML-страницы.
examples/	Малые модели и скрипты компиляции для примеров.
tests/	Тесты разбора, квантования, генерации, примеров, демонстрации и визуализации.
tools/eda-bin/	Локальные обертки для EDA-инструментов на Windows, если они есть.
docs/	Сценарий демонстрации, редактируемое руководство, PDF и локальный сборщик PDF.

6 Главные модули

Главная точка входа для пользователя находится в `torch2rtl/cli.py`. Команды устроены тонкими обертками над функциями пакета: это облегчает тестирование и позволяет примерам вызывать те же функции напрямую.

`torch2rtl/frontend/pytorch_fx.py` загружает модель из файла и ожидает, что файл определит `create_model()` или глобальную переменную `model`. После загрузки модель переводится в режим оценки, если у нее есть метод `eval`.

```
def load_model_from_file(path: Path) -> object:
    module = _load_python_module(path)
    if hasattr(module, "create_model"):
        model = module.create_model()
    elif hasattr(module, "model"):
        model = module.model
    else:
        raise ValueError(
            f"{path} must define create_model() or a global variable named
            ↪ model"
        )
    if hasattr(model, "eval"):
        model.eval()
    return model
```

6.1 Автоматический fusion BatchNorm2d

`load_model_from_file()` переводит загруженную модель в `eval`. При прямом вызове `parse_model()` это должен сделать пользователь. `parse_model()` не меняет режим исходной модели.

```
import torch.nn as nn

from torch2rtl.frontend.pytorch_fx import parse_model

model = nn.Sequential(
    nn.Conv2d(3, 8, kernel_size=3, padding=1, bias=False),
    nn.BatchNorm2d(8, affine=True),
    nn.ReLU(),
).eval()

graph = parse_model(model, input_shape=(3, 16, 16))
```

Публичная форма входа остается (C, H, W). После построения FX-графа frontend находит точную соседнюю пару `nn.Conv2d->nn.BatchNorm2d` и вызывает официальный `torch.nn.utils.fusion.fuse_conv_bn_eval()`. Для каждой допустимой пары в FX-графе создается отдельный `Conv2d` с пересчитанными весами и новым смещением `bias`. Исходная модель не меняется. `BatchNorm2d` не появляется в GraphIR, фиксированно-точечной эталонной модели или RTL.

В режиме `eval` `BatchNorm2d` использует фиксированную накопленную статистику. Поэтому его поканальное преобразование можно включить в веса и смещение `Conv2d` без отдельного оператора и отдельного аппаратного блока.

Для одного выходного канала fusion использует формулы

```
scale = gamma / sqrt(running_var + eps)
W' = W * scale
b' = (b - running_mean) * scale + beta
```

Если у `Conv2d` нет смещения, в формуле берется `b=0`. При `BatchNorm2d(affine=False)` используются `gamma=1` и `beta=0`. Если API установленной версии PyTorch позволяет задать `BatchNorm2d(affine=True, bias=False)`, параметр `gamma`

сохраняется, а `beta=0`. Frontend пересчитывает каждую пару отдельно, поэтому цепочка с несколькими парами получает независимые `W'` и `b'`.

Пакет размера `N=1` нужен только для Boundary A. На этой границе исходная PyTorch-модель получает `NCHW`, а FX GraphModule после fusion получает `CHW`. После проверки GraphIR и весь аппаратный путь снова работают только с `CHW`. Это не означает, что публичный интерфейс поддерживает пакетную обработку.

Поддерживаются четыре сочетания `Conv2d.bias` и `BatchNorm2d.affine`, а также `float32` и `float64`. При `affine=True` weight должен иметь точный тип `nn.Parameter`. `bias` может быть `None` только в версии PyTorch, чей конструктор принимает `bias=False`; в остальных случаях `bias` должен иметь точный тип `nn.Parameter`. При `affine=False` оба слота должны быть `None`.

`eps` должен иметь встроенный тип `float` и конечное неотрицательное значение. Значение `eps=0.0` допустимо, если выражение `running_var+eps` имеет конечное положительное значение в каждом канале. Отрицательный `eps` отклоняется до вызова официального fusion. Для fusion также требуются точные типы модулей, режим `eval`, корректная накопленная статистика, совпадающее число каналов и отсутствие fan-out. `BatchNorm2d` вне пары, режим `training`, `track_running_stats=False`, подклассы, `BatchNorm1d` и `BatchNorm3d` приводят к `UnsupportedOpError`.

`Flatten` после fusion разрешается только при структурном совпадении. После применения `Flatten` должно выполняться `source_flatten_shape==(1,*fused_flatten_shape)`. Поэтому `Flatten(-3, -1)` после `Conv2d` допустим, а `Flatten(0, -1)` нет, даже если число элементов случайно совпадает.

При fusion в `GraphIR.metadata` появляются два поля:

```
graph.metadata["input_adapter"]
# {"kind": "singleton_batch_n1"}

graph.metadata["transformations"]
# [{
#   "kind": "conv2d_batchnorm2d_fusion",
#   "conv_node": "conv",
#   "conv_target": "conv",
#   "batchnorm_node": "bn",
#   "batchnorm_target": "bn",
#   "fused_target": "_torch2rtl_fused_conv_bn_0",
# }]
```

Компилятор записывает те же необязательные поля в `report.json` по путям `graph.input_adapter` и `graph.transformations`. У модели без fusion этих полей нет. Метрика `metrics.parameters` учитывает параметры нового `Conv2d`, включая смещение `bias`, которое появляется при fusion исходного `Conv2d(bias=False)`.

Boundary A сравнивает выход исходной модели с выходом FX GraphModule после fusion. Форма, dtype и конечность значений проверяются отдельно. После этого frontend требует точного совпадения `argmax`. Изменение итогового класса при близких значениях логитов всегда приводит к `UnsupportedOpError`, даже если сами логиты проходят `allclose` с допуском для своего dtype.

Внутреннее представление задано короткими замороженными классами. Фрагмент из `torch2rtl/ir/ops.py` показывает операции, с которыми дальше работают квантование и генератор.

```
@dataclass(frozen=True)
```

```
class LinearIR:
    name: str
    input: TensorIR
    output: TensorIR
    in_features: int
    out_features: int
    weight: np.ndarray
    bias: np.ndarray | None

@dataclass(frozen=True)
class ReluIR:
    name: str
    input: TensorIR
    output: TensorIR
```

`torch2rtl/backend/systemverilog/emit.py` отвечает за создание выходного каталога, очистку старых сгенерированных файлов, квантование графа, запись проверочных векторов, применение шаблонов и запись отчетов. Эта функция возвращает `BuildReport`, чтобы вызывающий код знал, какие файлы появились.

```
def emit_systemverilog(
    graph: GraphIR,
    cfg: FixedPointConfig,
    out_dir: Path,
    vector_count: int = 16,
    seed: int = 0,
) -> BuildReport:
    out_dir.mkdir(parents=True, exist_ok=True)
    _clear_previous_outputs(out_dir)
    qgraph = quantize_graph(graph, cfg)
    vector_files = write_vector_files(out_dir, qgraph, cfg, vector_count, seed)
```

`torch2rtl/verify/simulator.py` выбирает симулятор. Если найдены `iverilog` и `vvp`, используется `Icarus Verilog`. Если их нет, но найден `verilator`, используется `Verilator`. Если нет ни одного варианта, статус записывается как `not_found`.

```
iverilog = find_eda_tool("iverilog")
vvp = find_eda_tool("vvp")
verilator = find_eda_tool("verilator")
if iverilog and vvp:
    result = _run_icarus(build_dir, iverilog, vvp)
elif verilator:
    result = _run_verilator(build_dir, verilator)
else:
    result = SimulationResult(
        ok=False,
        status="not_found",
        message="simulator not found: install Icarus Verilog or Verilator",
    )
```

`torch2rtl/synth/yosys.py` делает простой проход Yosys: читает `SystemVerilog`, выполняет `prep`-топтор и `stat`. Это не полный маршрут до платы, а быстрый отчет о структуре после синтезирующего анализа.

7 Пошаговый разбор демонстрационного примера

Для разбора выбран пример `tiny-conv`, потому что он небольшой, не требует предварительного обучения и проходит через все основные виды операций: `Conv2d`, `ReLU`, `Flatten`, `Linear`, `Argmax`.

Демонстрация объявлена в `torch2rtl/demo.py`.

```
"tiny-conv": DemoSpec(  
    name="tiny-conv",  
    model_path=REPO_ROOT / "examples" / "tiny_conv" / "model.py",  
    input_shape=(1, 3, 3),  
    description="Conv2d/ReLU/Linear classifier for the smallest CNN path.",  
    seed=11,  
),
```

Модель находится в `examples/tiny_conv/model.py`. Ее основная последовательность:

```
self.net = nn.Sequential(  
    nn.Conv2d(  
        INPUT_CHANNELS,  
        CONV_CHANNELS,  
        kernel_size=2,  
    ),  
    nn.ReLU(),  
    nn.Flatten(start_dim=0),  
    nn.Linear(CONV_CHANNELS * CONV_ROWS * CONV_COLS, CLASS_COUNT),  
)
```

Вход имеет форму $(1, 3, 3)$: один канал, три строки, три столбца. Свертка с ядром 2×2 без дополнения дает выход $(1, 2, 2)$. После `Flatten(start_dim=0)` четыре значения становятся вектором длины 4. Последний слой `Linear(4, 4)` оставляет четыре логита. Это настоящий выход PyTorch и GraphIR. `Hardware-lowering` дополнительно вычисляет `class_id`, не подменяя логиты.

Весы в примере заданы вручную, а не обучены. Фрагмент из `examples/tiny_conv/model.py`:

```
conv.weight.zero_()  
conv.bias.zero_()  
conv.weight[0, 0] = torch.tensor(  
    [  
        [0.50, 0.25],  
        [0.25, 0.50],  
    ],  
    dtype=conv.weight.dtype,  
)  
  
linear.weight.zero_()  
linear.bias.zero_()  
for class_idx in range(CLASS_COUNT):  
    linear.weight[class_idx, class_idx] = 1.0
```

Здесь свертка взвешивает каждый квадрат 2×2 , а линейный слой фактически выбирает один из четырех элементов результата. Такая модель хорошо подходит для демонстрации: из нее легко мысленно проследить, почему конкретный участок входной сетки дает большой логит.

Разбор PyTorch-графа происходит в `parse_model`. Для `Conv2d` код проверяет форму входа, группы, режим дополнения и растяжение ядра. Фрагмент из `torch2rtl/frontend/pytorch_fx.py`:

```
if isinstance(module, nn.Conv2d):
    if len(current_tensor.shape) == 4:
        raise UnsupportedOpError(
            "Unsupported Conv2d input: batch dimension is not supported; "
            "use unbatched (C, H, W)"
        )
    if len(current_tensor.shape) != 3:
        raise UnsupportedOpError(
            f"Unsupported Conv2d input rank: expected (C, H, W), got "
            ↪ {current_tensor.shape}"
        )
    if int(module.groups) != 1:
        raise UnsupportedOpError("Unsupported Conv2d groups: only groups=1 "
            ↪ is supported")
```

После разбора получается последовательность операций. Тест `tests/test_examples.py` фиксирует ожидаемый порядок для `tiny-conv`:

```
assert [type(op) for op in graph.ops] == [
    Conv2dIR,
    ReluIR,
    FlattenIR,
    LinearIR,
]
```

Далее генератор создает файлы в выходном каталоге. Для `tiny-conv` ожидаются, в частности, `conv2d_comb.sv`, `linear_comb.sv`, `relu.sv`, `argmax.sv`, `top.sv`, `tb_top.sv`, `input_vectors.txt`, `expected_classes.txt`, `expected_logits.txt`, файлы весов и смещений, `report.json`, `visualization.json`, `visualization.html`.

В `visualization.json` та же структура становится блоками. Тест `tests/test_examples.py` проверяет порядок основных блоков:

```
assert [block["kind"] for block in manifest["blocks"] if block["lane"] ==
    ↪ "main"] == [
    "input",
    "conv2d",
    "relu",
    "flatten",
    "linear",
    "argmax",
    "output",
]
```

Почему это важно: один пример связывает все уровни проекта. По нему видно, как класс PyTorch превращается в список операций, затем в фиксированно-точечный граф, затем в SystemVerilog-модули и в страницу визуализации.

8 Фиксированная точка

В проекте число с фиксированной точкой хранится как целое. Параметр `frac_bits` задает, сколько младших битов относится к дробной части. При значении `frac_bits=6` масштаб равен 64: вещественное значение 1.0 становится целым 64,

значение 0.5 становится 32.

Конфигурация описана в `torch2rtl/quant/fixed_point.py`.

```
@dataclass(frozen=True)
class FixedPointConfig:
    bits: int = 8
    frac_bits: int = 6
    acc_bits: int = 32

    def __post_init__(self) -> None:
        # Полная реализация также проверяет integer-типы, исключая bool.
        if not 2 <= self.bits <= 32:
            raise ValueError("bits must be in the supported signed range 2..32")
        if not 0 <= self.frac_bits < self.bits:
            raise ValueError("frac_bits must satisfy 0 <= frac_bits < bits")
        if not self.bits < self.acc_bits <= 64:
            raise ValueError("acc_bits must satisfy bits < acc_bits <= 64")
```

Квантование округляет значение после умножения на масштаб, затем применяет насыщение:

```
def quantize_array(values: np.ndarray, cfg: FixedPointConfig) -> np.ndarray:
    source = np.asarray(values)
    # Реализация принимает float32/float64, проверяет finite и сохраняет
    # исходный floating dtype при умножении и ties-to-even rounding.
    # Overflow конечного float при масштабировании означает saturation.
    with np.errstate(over="ignore"):
        scaled = np rint(source * np.asarray(cfg.scale, dtype=source.dtype))
    return saturate_array(scaled.astype(np.float64), cfg.bits).astype(np.int64)
```

Для слоя Linear эталонный расчет на Python сначала переносит смещение в разрядность аккумулятора, затем накапливает произведения и сдвигает результат обратно на число дробных битов. Фрагмент из `torch2rtl/quant/reference.py`:

```
for out_idx in range(w.shape[0]):
    acc = int(b[out_idx]) << cfg.frac_bits
    for in_idx in range(w.shape[1]):
        acc += int(x[in_idx]) * int(w[out_idx, in_idx])
    shifted = acc >> cfg.frac_bits
    outputs.append(saturate_int(shifted, cfg.bits))
```

Та же идея повторяется в шаблонах SystemVerilog. Фрагмент из `torch2rtl/back end/systemverilog/templates/linear_comb.sv.j2`:

```
always_comb begin
    for (out_idx = 0; out_idx < OUT_FEATURES; out_idx = out_idx + 1) begin
        acc = extend_q(bias_at(biases, out_idx)) <<< FRAC_BITS;
        for (in_idx = 0; in_idx < IN_FEATURES; in_idx = in_idx + 1) begin
            acc = acc
                + (extend_q(input_at(in_data, in_idx))
                  * extend_q(weight_at(weights, out_idx*IN_FEATURES +
                    ↪ in_idx)));
        end
        out_data[out_idx*DATA_BITS +: DATA_BITS] = sat_q(acc >>> FRAC_BITS);
    end
end
```

Почему смещение сдвигается влево: входы и веса уже содержат множитель масштаба. Их произведение содержит масштаб дважды. Сумма произведений хранится в аккумуляторе, затем сдвиг вправо возвращает результат к исходному масштабу. Смещение перед сложением переводится в тот же масштаб, что и произведения в аккумуляторе.

Тест `tests/test_quant.py` подтверждает насыщение для 8 бит и 6 дробных битов:

```
def test_quantization_clamps_correctly() -> None:
    cfg = FixedPointConfig(bits=8, frac_bits=6, acc_bits=32)
    values = np.asarray([-10.0, -2.0, 0.0, 1.0, 10.0])
    quantized = quantize_array(values, cfg)
    assert quantized.tolist() == [-128, -128, 0, 64, 127]
```

9 Создаваемые SystemVerilog-модули

Генератор создает несколько файлов, часть из них постоянна для любой поддерживаемой модели, а часть зависит от наличия операций и параметров.

Файл	Роль
<code>top.sv</code>	Верхний модуль. Соединяет слои, объявляет параметры разрядности, входную шину, выходной номер класса и логиты.
<code>conv2d_comb.sv</code>	Комбинационная свертка. Нужна, если граф содержит <code>Conv2dIR</code> .
<code>linear_comb.sv</code>	Комбинационный полносвязный слой.
<code>relu.sv</code>	Позлементная отсечка отрицательных значений.
<code>argmax.sv</code>	Выбор индекса максимального логита.
<code>tb_top.sv</code>	Проверочный модуль, который читает входы и побитово сравнивает ожидаемые классы и logits.
<code>*_weights.mem</code>	Целочисленные веса слоя после квантования.
<code>*_bias.mem</code>	Целочисленные смещения слоя после квантования.
<code>input_vectors.txt</code>	Сгенерированные входные векторы.
<code>expected_classes.txt</code>	Ожидаемые классы по фиксированно-точечной эталонной модели.
<code>expected_logits.txt</code>	Ожидаемые signed logits по фиксированно-точечной эталонной модели.
<code>report.json</code>	Отчет о графе, параметрах, метриках, инструментах и проверках.
<code>visualization.json</code>	Данные для страницы визуализации.
<code>visualization.html</code>	Самодостаточная страница визуализации.

Верхний модуль строится из списка слоев. Фрагмент из `torch2rtl/backend/systemverilog/templates/top.sv.j2`:

```
module top #(
    parameter int DATA_BITS = {{ data_bits }},
    parameter int FRAC_BITS = {{ frac_bits }},
    parameter int ACC_BITS = {{ acc_bits }},
```

```

parameter int INPUT_SIZE = {{ input_size }},
parameter int CLASS_COUNT = {{ class_count }},
parameter int CLASS_BITS = {{ class_bits }}
) (
  input logic signed [INPUT_SIZE*DATA_BITS-1:0] in_data,
  output logic [CLASS_BITS-1:0] class_id,
  output logic signed [CLASS_COUNT*DATA_BITS-1:0] logits
);

```

Модуль `argmax` выбирает первый максимум, потому что обновляет `class_id` только при строгом сравнении `candidate > best_value`. Фрагмент из `torch2rtl/backend/systemverilog/templates/argmax.sv.j2`:

```

always_comb begin
  best_value = input_at(in_data, 0);
  class_id = '0;
  for (idx = 1; idx < FEATURES; idx = idx + 1) begin
    candidate = input_at(in_data, idx);
    if (candidate > best_value) begin
      best_value = candidate;
      class_id = idx[CLASS_BITS-1:0];
    end
  end
end
end

```

Проверочный модуль знает точное положительное число векторов, читает ровно это число записей и после цикла проверяет EOF каждого файла. Фрагмент из `torch2rtl/backend/systemverilog/templates/tb_top.sv.j2`:

```

for (vector_idx = 0; vector_idx < EXPECTED_VECTORS; vector_idx =
↪ vector_idx + 1) begin
  status = $fscanf(fd_expected, "%s", token);
  if (status != 1) begin
    $display("FAIL vector=%0d missing expected class", vector_idx);
    errors = errors + 1;
  end else if ($sscanf(token, "%d", expected_class) != 1 ||
    token != $sformatf("%0d", expected_class)) begin
    $display("FAIL vector=%0d malformed expected class",
    ↪ vector_idx);
    errors = errors + 1;
  end else if (expected_class < 0 || expected_class >= CLASS_COUNT)
    ↪ begin
    $display("FAIL vector=%0d expected class out of range: %0d",
    ↪ vector_idx, expected_class);
    errors = errors + 1;
  end
  for (idx = 0; idx < INPUT_SIZE; idx = idx + 1) begin
    status = $fscanf(fd_inputs, "%s", token);
    if (status != 1) begin
      $display("FAIL vector=%0d input=%0d missing input value",
      ↪ vector_idx, idx);
      errors = errors + 1;
      tmp = 0;
    end else if ($sscanf(token, "%d", tmp) != 1 ||
      token != $sformatf("%0d", tmp)) begin
      $display("FAIL vector=%0d input=%0d malformed value",
      ↪ vector_idx, idx);
      errors = errors + 1;
    end
  end
end

```

```

        tmp = 0;
    end else if (tmp < DATA_MIN || tmp > DATA_MAX) begin
        $display("FAIL vector=%0d input=%0d value out of range:
        ↪ %0d", vector_idx, idx, tmp);
        errors = errors + 1;
    end
    in_data[idx*DATA_BITS +: DATA_BITS] = tmp[DATA_BITS-1:0];
end
#1;
if (class_id !== expected_class[CLASS_BITS-1:0]) begin
    $display("FAIL vector=%0d expected=%0d got=%0d", vector_idx,
    ↪ expected_class, class_id);
    errors = errors + 1;
end

```

Каждое значение сначала читается в `string` и принимается только если 64-битный signed %d разбор обратимо даёт ту же каноническую decimal-строку. Поэтому даже $\text{value} + 2^{64}$ не может обернуться в исходное значение. `tmp`, `expected_class` и `expected_logit` затем проверяются по диапазону до среза аппаратной шины. После цикла отдельные чтения отвергают лишние или malformed значения в `input`, `classes` и `logits`. Host-validator сверяет metadata с единственными активными `localparam`, лексически различая SystemVerilog-код, строки и line/block comments. Поэтому `localparam` внутри строки или комментария не может подменить реально скомпилированный контракт. Preprocessor directives запрещены, а сами параметры должны находиться в строгом generated header сразу после `module tb_top`, не во вложенной области. Runner принимает только единственную точную строку `PASS vectors=N`, где $N > 0$ и совпадает с metadata.

Текущие SystemVerilog-модули описывают комбинационную схему. Это видно по интерфейсу `top.sv`: нет тактового входа, сброса, сигналов готовности или потокового протокола. Такой выбор делает результат проще для чтения и проверки, но накладывает ограничения на размер моделей и на практическое использование.

10 Установка и запуск

Release v0.2 поддерживает только Python $\geq 3.12, < 3.13$. Python 3.11 и 3.13 не входят в подтверждённую границу поддержки. В `README.md` рекомендован запуск через `uv`:

```
uv --cache-dir temp/uv-cache sync --dev
```

В текущем состоянии `uv.lock` закрепляет CPU-сборку PyTorch. Это уменьшает объем установки на Linux-ноутбуке и делает демонстрационный запуск воспроизводимее, чем lock-файл с CUDA-зависимостями.

Компиляция модели через CLI:

```
uv --cache-dir temp/uv-cache run torch2rtl compile examples/tiny_conv/model.py
↪ --input-shape 1 3 3 --out build/tiny_conv
```

Та же идея через скрипт примера:

```
uv --cache-dir temp/uv-cache run python examples/tiny_conv/compile.py --out
↪ build/tiny_conv
```

Демонстрационная команда для полного локального маршрута без платы:

```
uv --cache-dir temp/uv-cache run torch2rtl demo --name tiny-conv --out  
↳ build/demo
```

Команда `demo` также принимает имя демонстрации позиционно:

```
uv --cache-dir temp/uv-cache run torch2rtl demo tiny-conv --out build/demo
```

Доступные демонстрации перечислены в `torch2rtl/demo.py`: `tiny-mlp`, `grid-classifier`, `tiny-conv`, `image-cnn`. Скрипты `compile` для обучаемых примеров загружают checkpoint: `tiny_mlp/compile.py` требует явный `--checkpoint`, а `image_cnn/compile.py` использует свой `checkpoint path`. Model-файлы этих четырёх демо включаются в `wheel`, поэтому та же команда `demo` работает и из установленного пакета без исходного checkout.

11 Проверка результата

После `compile` главный признак успешной генерации: в выходном каталоге появились `top.sv`, `tb_top.sv`, `report.json`, `visualization.json` и `visualization.html`. Тест `tests/test_sv_generation.py` проверяет наличие ключевых модулей:

```
assert "top.sv" in report.generated_files  
assert "tb_top.sv" in report.generated_files  
assert (tmp_path / "top.sv").exists()  
assert (tmp_path / "report.json").exists()
```

Моделирование запускается командой:

```
uv --cache-dir temp/uv-cache run torch2rtl verify build/tiny_conv
```

Если доступен Icarus Verilog, проект компилирует источники командой вида `iv erilog-g2012-osimv...`, затем запускает `vvp`. Если Icarus Verilog не найден, но найден Verilator, используется Verilator. Если симулятор не найден, явная команда `verify` возвращает ненулевой код. Только `demo` может обозначить внешний EDA-этап как `optional/skipped`.

Успех моделирования определяется точной строкой `PASSvectors=N` с $N > 0$, без строк `FAIL` и с совпадением `N` с `metadata`. Это зафиксировано в `tests/test_simulator_runner.py`:

```
def test_simulation_ok_requires_pass_without_failures() -> None:  
    assert _simulation_ok(0, "PASS vectors=16\n")  
    assert not _simulation_ok(0, "PASS vectors=0\n")  
    assert not _simulation_ok(0, "FAIL vector=16 expected=2 got=1\nFAIL  
↳ errors=1\n")  
    assert not _simulation_ok(1, "PASS vectors=16\n")
```

Синтезирующий анализ запускается командой:

```
uv --cache-dir temp/uv-cache run torch2rtl synth build/tiny_conv
```

Если `yosys` найден, создается `yosys.log`, а метрики попадают в `report.json`. Если `yosys` не найден, отчет получает статус `not_found`.

Визуализация строится или обновляется командой:

```
uv --cache-dir temp/uv-cache run torch2rtl visualize build/tiny_conv  
↪ --vector-index 0
```

HTML-страница самодостаточна: в нее встроены данные, стили и сценарий. Это подтверждается тестом `tests/test_visualization.py`, который ищет в файле `manifest-data`, `circuit-root`, `Linear/MAC` и `Torch2RTL`.

12 Проведение демонстрации

Для живого показа лучше выбрать `tiny-conv`: он достаточно мал, чтобы объяснить все руками, и достаточно богат, чтобы показать свертку, фиксированную точку, проверочные данные и визуализацию.

Рекомендуемый сценарий:

- Показать `examples/tiny_conv/model.py` и форму входа $(1, 3, 3)$.
- Объяснить ядро свертки 2×2 и почему выход имеет форму $(1, 2, 2)$.
- Запустить `torch2rtldemo --name tiny-conv --out build/demo`.
- Открыть `build/demo/report.json` и показать список операций, параметры фиксированной точки и статусы инструментов.
- Открыть `build/demo/top.sv` и показать, как слои соединены сигналами.
- Открыть `build/demo/tb_top.sv` и показать чтение `input_vectors.txt`, `expected_classes.txt` и `expected_logits.txt`.
- Открыть `build/demo/visualization.html` в браузере и пройти по блокам схемы.

Если симулятор или Yosys не установлены, это не обязательно ломает показ. Код `demo` считает такие статусы допустимыми: свойство `ok` в `DemoResult` принимает `not_found` как нефатальный результат для моделирования и синтеза.

```
@property  
def ok(self) -> bool:  
    simulation_ok = self.simulation.ok or self.simulation.status ==  
        ↪ "not_found"  
    synthesis_ok = self.synthesis.ok or self.synthesis.status == "not_found"  
    return simulation_ok and synthesis_ok
```

При этом стоит проговорить аудитории разницу: генерация SystemVerilog и HTML-отчета прошла, но внешние EDA-инструменты не подтвердили схему, если их нет в окружении.

13 Ограничения

Ограничения ниже подтверждены кодом разбора, тестами и текущими шаблонами:

- нет поддержки размерности пакета данных для `Conv2d`;
- `Linear` поддерживает только 1-D вход без `batch/prefix dimensions`;
- `Flatten` должен превращать весь тензор в один вектор;
- `fusion` поддерживает только точную пару `Conv2d->BatchNorm2d` в `eval` с

накопленной статистикой, без BatchNorm2d вне пары и без fan-out;

- BatchNorm2d(affine=True, bias=False) требует версии PyTorch с аргументом bias; конечный неотрицательный eps разрешен только при конечном положительном running_var+eps в каждом канале;

- при fusion Flatten должен сохранять структуру пакета размера 1 по правилу `source_flatten_shape==(1,*fused_flatten_shape)`;

- FX-граф должен быть одной цепочкой без ветвлений и нескольких выходов;

- explicit argmax поддерживан только с dim=None, keepdim=False;

- dtype параметров и buffers должен быть однородным float32 или float64;

- module/global hooks, subclasses, custom metaclasses, instance forward/call

overrides, custom call/attribute/copy descriptors, decorated forward и переопределённые module-introspection API отклоняются для любого вложенного модуля;

- custom Python bytecode вне закрытого подмножества, nested code objects,

conditions, loops, exception handling, FX Proxy introspection и tensor-dependent formatting отклоняются до trace;

- class-level/external Python state и module-owned state, изменяемое во время FX

trace или concrete semantic probe, отклоняются; обе проверки выполняются на копиях и не меняют исходную модель;

- параметры/buffers должны быть точными Parameter/Tensor с обычными

string-именами на CPU, с layout strided и contiguous storage; tensor subclasses, negative/conjugate view bits, сохранённый .grad, прямое tensor/ndarray instance-state и доступ custom forward к внутренним registries отклоняются;

- monkeypatch стандартных PyTorch classes, forward definitions и functional

kernels, а также FX graph/tracer classes не входит в поддерживаемую семантику;

- ReLU(inplace=True) отклоняется, потому что RTL не моделирует мутацию входа;

- нет динамического управления исполнением модели;

- форма входа должна быть известна при запуске compile;

- input shape должна быть конечным Sequence с rank ≤ 64 и не более

1 000 000 элементов; output Conv2d имеет тот же element limit;

- нет обучения, обратного распространения ошибки и логики GPU внутри компилятора;

- для Conv2d поддерживаны только groups=1, нулевое дополнение и dilation=1;

- kernel_size/stride должны быть положительными integer, padding —

неотрицательным, формы параметров обязаны совпадать с атрибутами, а structural values и максимальная координата должны помещаться в signed 32-bit SystemVerilog int;

- structural sizes должны быть положительными built-in int; input dimensions и Flatten.start_dim/end_dim не преобразуют bool, строки или subclasses;
- нет grouped convolution, depthwise convolution, других операций нормализации, attention и больших современных архитектур;
- текущая схема комбинационная и не имеет потокового интерфейса;
- Yosys используется только для простого read_verilog, prep-toptop, stat;
- проверочные векторы случайны и строятся для фиксированно-точечной эталонной модели, а не для полного набора входов; перед EDA metadata сверяется с полностью каноническим testbench, top.sv и report, а input/logits/class проверяются по разрядным диапазонам; simulation-control constructs в DUT sources запрещены вне testbench, SHA-256 manifest привязывает каждый RTL source к compile output, а synth preflight сверяет report widths/sizes с top/TB/vector metadata без чтения vector payload; synthesis result не теряется, если подробный visualization trace недоступен из-за повреждённых векторов;
- эти проверки обнаруживают случайное повреждение проверяемого generated contract;
- одиночная несогласованная правка RTL, testbench, vector или обязательной report metadata даёт preflight либо bit-exact failure;
- SHA-256 гарантированно связывает только RTL sources, а для vectors и report проверяются структура, диапазоны и contract-bearing metadata;
- обнаружение произвольной допустимой замены vector payload без изменения результата или несвязанного report-поля не гарантируется;
- локальный manifest хранится в том же build directory и не является внешним корнем доверия;
- согласованная злонамеренная замена RTL, vectors, report и хешей одновременно находится вне threat model;
- защита от такого сценария потребовала бы внешней подписи, доверенного manifest либо полной регенерации из доверенных исходников;
- качество классификации зависит от выбранной модели и квантования; проект не доказывает точность модели на реальном наборе данных.

Если граф содержит неподдержанный узел, parse_model выбрасывает UnsupportedOpError. Это лучше, чем молча сгенерировать неправильную схему: пользователь сразу видит, на какой операции остановился разбор.

14 Дальнейшее развитие

В README.md указан следующий маршрут развития:

Версия	Направление
v0.1	Linear, ReLU, Flatten, Argmax.
v0.2	Semantic Correctness и воспроизводимый board-free release gate.

Версия	Направление
v0.3	Проверка ассурасы обученной CNN по цепочке float → fixed → RTL.
v0.4	Последовательный генератор MAC.
v0.5	Потоковый интерфейс или интерфейс, похожий на AXI.

Практически это означает несколько инженерных направлений:

- добавить последовательный вариант арифметики, чтобы уменьшить количество параллельных умножителей;
- расширить набор поддерживаемых операций PyTorch;
- сделать интерфейс `top.sv` ближе к реальному аппаратному блоку: такт, сброс, готовность входа и выхода;
- улучшить оценку погрешности между вещественной и фиксированно-точечной моделью;
- расширить демонстрационные модели, сохранив их малый размер и объяснимость;
- добавить больше тестов на крайние случаи квантования, свертки и форм тензоров.

Главный принцип для развития проекта: не терять прозрачность. Новая возможность должна быть видна во внутреннем представлении, покрыта эталонным расчетом и иметь небольшой пример, по которому можно проверить всю цепочку от PyTorch до SystemVerilog.